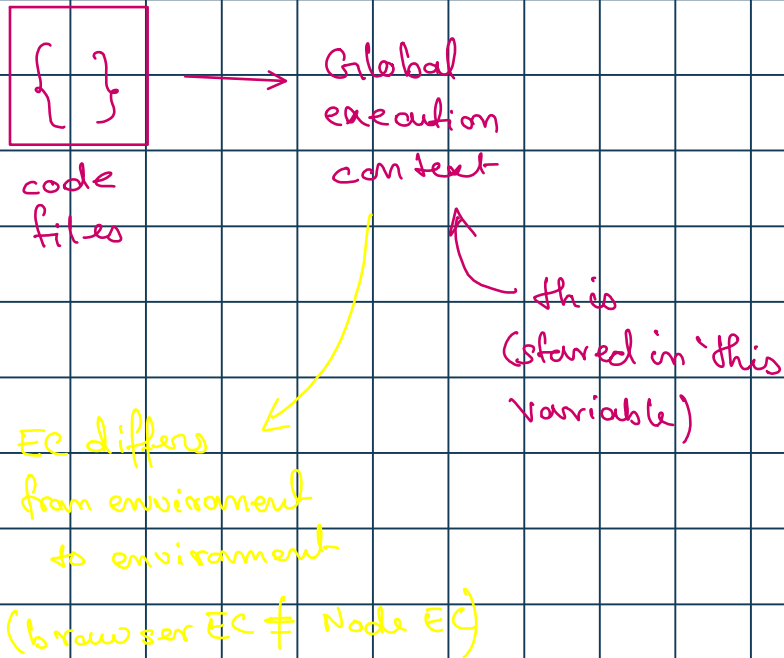


How Javascript executes code + call stack

02 February 2026

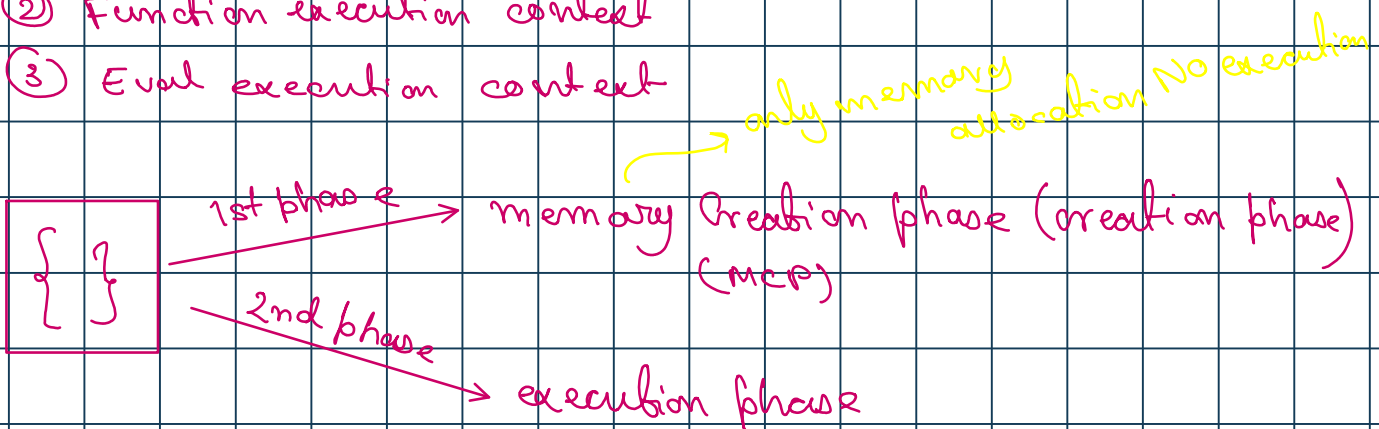
08:06 PM

JAVASCRIPT EXECUTION CONTEXT



- Everything executes in a thread
- Javascript is single threaded (treated as a process)

- ① Global execution context
- ② Function execution context
- ③ Eval execution context



```
1 → let val1 = 10
2 → let val2 = 7
3
4 → function addNum(num1, num2){
5     let total = num1 + num2
6     return total
}
```

```

4 → function addNum(num1, num2){
5     let total = num1 + num2
6     return total
7 }
8
9 let result1 = addNum(val1, val2)
10 let result2 = addNum(20, 30)

```

Step 1.

Global Execution/Environment

↓
allocated in this

Step 2

Memory Phase

val1 → undefined
val2 → undefined
addNum → definition
result1 → undefined
result2 → undefined

first cycle

Step 3

Execution Phase

val1 = 10
val2 = 7
addNum →
result1 = 15
result2 = 50

Execution context

new
variable
environment
+
execution
thread

(delete after
execution)

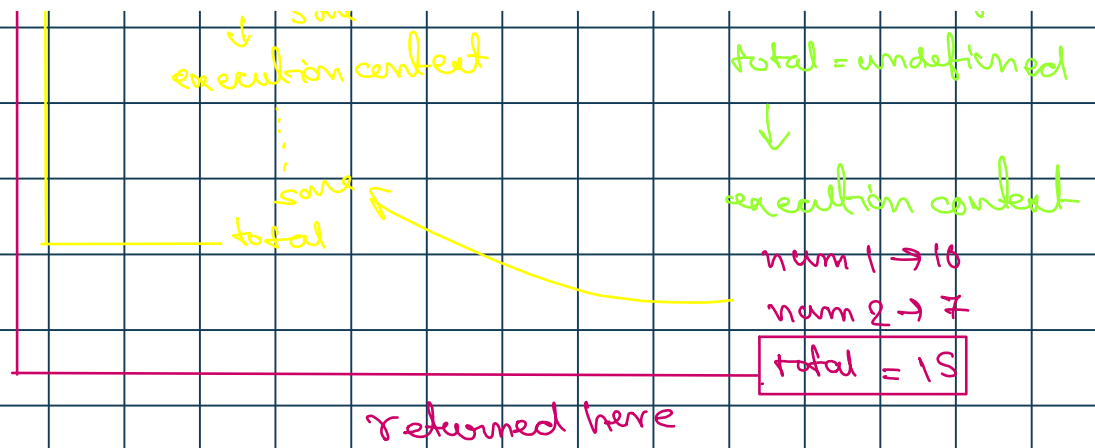
NVE
thread

memory phase

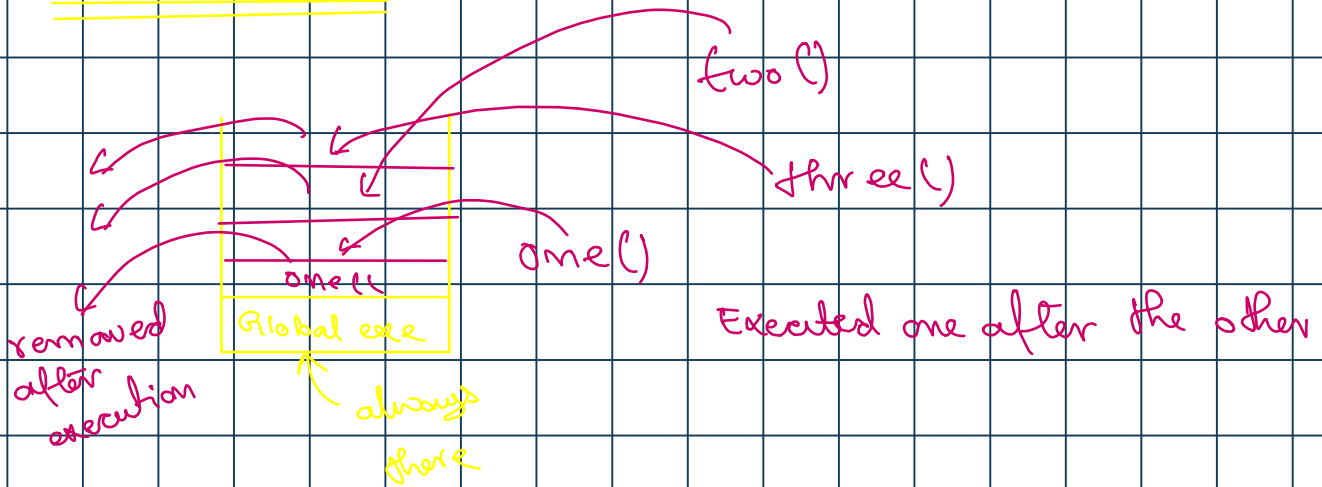
↓
same
↓
execution context

memory execution phase

val1 = undefined
val2 = undefined
total = undefined



CALL STACK



BUT if you have nested functions:

```

one() {
  two() {
    three()
  }
}

```

